

Trigrams & sparsity

Episode 02A | Theory companion guide

Building an LLM From Scratch - Hussain Abuwala

Our first model remembered one character. What changes when it remembers two? More context can separate useful patterns, but a count table needs enough matching examples for each separate context.

This guide follows the theory video. The separate coding episode implements the models and measures their behavior on a larger names dataset. No Python, calculus, or matrix operations are required here.

The question changes; the task stays the same

A language model predicts a probability distribution over the next token. Here a token is a character, or the special END marker.

Model	Previous tokens used	Next-token question
Bigram	1	What follows n?
Trigram	2	What follows an?
4-gram	3	What follows ann?

The number in n-gram includes the target: a trigram uses **two context tokens plus one target**. It does not predict three characters at once.

Why two START tokens?

Before the first character, the trigram has two empty history positions. We fill both with the same START token, written S. E means END.

```
Bigram:    S a n n a E
Trigram:  S S a n n a E
```

The pads supply history; they are never prediction targets. Both models predict the four characters of anna, then END: **five observations**. Reset the history before each name.

How to use the materials

Read this guide alongside the 29-frame canvas. The canvas PDF is a portable reading copy; the Excalidraw file is editable. Download the HTML reader and open it locally for slide navigation and optional speaker notes. GitHub displays HTML source rather than running the reader.

Repository and all companion files:

<https://github.com/HussainAbuwala/llm-from-scratch>

1. Build the trigram table

Our original training corpus is **anna and ava**. Allowed next-token outcomes are **a, n, v, E**: four outcomes. START is an input marker, not an output.

```
anna: SS -> a, Sa -> n, an -> n, nn -> a, na -> E
ava:  SS -> a, Sa -> v, av -> a, va -> E
```

Count each observation in its context row:

Context	a	n	v	E	Total
SS	2	0	0	0	2
Sa	0	1	1	0	2
an	0	1	0	0	1
nn	1	0	0	0	1
na	0	0	0	1	1
av	1	0	0	0	1
va	0	0	0	1	1

Seven observed rows hold nine observations: five from anna, four from ava. A context is a tuple of tokens; writing SS or an is just compact notation here.

Turn counts into probabilities

For a seen context h , divide the target count by that row's total. $C(h,t)$ counts target t after context h ; $C(h)$ is the row total.

$$P(t | h) = \frac{C(h,t)}{C(h)}$$

For Sa, the n and v counts are both one out of two, giving one-half each. For an, the only observed target is n, giving n probability one.

Generate one token at a time

Start at SS, choose a next token using the row's probabilities, keep the latest two tokens as context, and repeat until END.

```
SS -> a -> choose n or v with equal probability
      n branch: then n, a, END -> anna
      v branch: then a, END   -> ava
```

This particular unsmoothed toy model generates only anna and ava, each with probability one-half. That restriction is a property of this learned table, not a rule that all language models only repeat their training examples.

2. Two kinds of missing evidence

Evaluation follows a supplied name's actual characters. It does not sample a new name. Consider **ana**, which was not in our training corpus.

Context	Required target	Raw probability
SS	a	1
Sa	n	1/2
an	a	0
na	E	1

Multiplying these probabilities gives zero. One impossible step makes the whole path impossible under this model.

$$P(\text{ana}) = 1 \times \frac{1}{2} \times 0 \times 1 = 0$$

An empty cell is different from an absent row

Query	What is missing?	Raw count result
a after an	The row exists, but a never followed it	0/1 = 0
a after vn	The entire context row is missing	0/0: no defined distribution

An all-zero row is not a probability distribution: its probabilities do not sum to one. For an unseen context, raw counts give us no preferred distribution. We must choose a policy, such as smoothing or fallback.

Connect this to negative log-likelihood

NLL measures how much probability the model assigns to the actual targets. Lower is better. For a complete name, divide the negative natural log of its path probability by its number of predictions, including END.

$$\text{NLL}(\text{ana}) = -\frac{\ln P(\text{ana})}{4}$$

A zero-probability path has infinite NLL. The Episode 01 bigram assigned ana probability 1/16, giving about 0.693147 nats per prediction. The raw trigram fails on this example.

This is one possible failure, not evidence that every trigram is worse than every bigram. We need held-out evaluation on more data to compare their practical performance.

3. Smoothing, with every count visible

Add-k smoothing adds the same positive pseudocount k to **every allowed outcome**, including outcomes whose counts were already positive.

$$P_k(t | h) = \frac{C(h, t) + k}{C(h) + kK}$$

K is the number of allowed next-token outcomes. Our original toy has $K = 4$. Pseudocounts are a modeling choice, not additional training observations.

Where does one-fifth come from?

Apply add-one smoothing to the an row:

Next token	Training count	Add one	Probability
a	0	1	1/5
n	1	2	2/5
v	0	1	1/5
E	0	1	1/5
Total	1	5	1

The denominator is one real observation plus four added counts. For an entirely unseen row, all four smoothed counts are one, so every outcome gets one-quarter.

Any positive k produces a uniform distribution for an entirely empty row. Smoothing cannot discover a preference from no observations.

Recompute the whole path

Apply the same smoothing rule to every step of ana, not just its failing step:

$$P_1(\text{ana}) = \frac{3}{6} \times \frac{2}{6} \times \frac{1}{5} \times \frac{2}{5} = \frac{1}{75}$$

$$\text{NLL}(\text{ana}) = -\frac{\ln(1/75)}{4} \approx 1.079372$$

We now have finite loss. But smoothing also redistributes probability away from observed outcomes; it does not guarantee a better score on every example.

Vocabulary matters: smoothing over a, n, v, E cannot assign a probability to an unknown character such as z. Unknown tokens require a separate vocabulary policy.

4. More detail, potentially less evidence

Longer context makes the question more specific. It does not create more training observations.

In *anna*, the bigram *n* row has two observations: *n* once and *a* once. The trigram separates these into *an* -> *n* and *nn* -> *a*, each observed once. Both trigram rows report 100% for their sole target.

100% from one observation describes the sample. It does not prove what must happen in future names.

Why can a longer row never have more matches?

Every *an* match is also an *n* match. Adding the condition that *a* must precede *n* can only keep or remove matches from the same data.

Dataset A: ...*an*...*an*...*bn*...*cn*...
 n: 4 matches *an*: 2 matches

Dataset B: ...*an*...
 n: 1 match *an*: 1 match

Dots are separators, not training characters. Keep each dataset fixed within its comparison. The longer context has **equal or fewer** matching observations than its shorter ending, called its suffix.

Continue beyond trigrams

For this example only, replace the corpus with **anna, enna, inna, onna**. These four constructed strings supply 20 predictions, including four END targets.

Context length	Context	Observations followed by a
2	<i>nn</i>	4
3	<i>ann, enn, inn, onn</i>	1 in each separate row

All these observed rows still predict *a* with probability one. Splitting evidence does not automatically make a prediction wrong; each estimate now has less support.

For a supplied history ending in **nnn**, the trigram uses *nn* and finds four observations. The 4-gram needs *nnn* and finds none. This is a supplied query, not a claimed generated sample.

This separate corpus has six outcomes: *a, e, i, n, o, E*. Its unseen smoothed row would give **1/6**, not the original toy's 1/4.

Longer context helps when the extra detail predicts something useful and the data provides enough examples to learn that distinction.

5. Why the table grows so quickly

With three ordinary characters, a, n, v, there are three one-character contexts, nine pairs, and 27 triples before considering START boundaries.

```
aa  an  av
na  nn  nv
va  vn  vv
```

Each additional position multiplies the possibilities by the alphabet size. With A ordinary characters and m context positions, there are A to the power m ordinary-character contexts.

Include valid START contexts

For two context positions and 26 ordinary letters:

Context family	Number of rows
SS	1
S followed by a letter	26
Two letters	$26 \times 26 = 676$
Total	703

START can only be a prefix. A context such as aS is invalid. END is a prediction target, never part of the history because generation stops there.

$$R_m = 1 + A + A^2 + \dots + A^m$$

Each row has $A + 1$ outcomes, including END. Full-table cells = rows times outcomes.

Context length	Model	Possible rows	Possible cells
1	Bigram	27	729
2	Trigram	703	18,981
3	4-gram	18,279	493,533
4	5-gram	475,255	12,831,885
5	6-gram	12,356,631	333,629,037

These are calculated capacities for $A = 26$, not measured training results. Some allowed events, such as END immediately after START padding, never appear in our nonempty training names but remain in the smoothed outcome space.

Meanwhile, each name still supplies its number of characters plus one END. Original toy: nine observations. Separate four-string corpus: twenty. **Possible contexts multiply while the training evidence stays fixed.**

6. Storage and evidence are different problems

A full array reserves space for every possible cell. A sparse dictionary stores only the counts observed during training.

For smoothing, a missing target count is treated as zero; an absent row has total zero. The model can calculate probabilities on demand for evaluation and generation, using the full allowed outcome vocabulary in the denominator.

There is no need to store every smoothed probability or insert unseen rows into the training dictionary.

Sparse storage saves memory. It does not supply evidence about what follows a missing context.

Responses and their limits

Response	Benefit	Limit
More representative data	More relevant matching examples	Long contexts may still be uncovered
Shorter context	Pools observations into broader groups	Can lose useful detail
Smoothing	Gives known outcomes positive probability	Empty rows still have no learned preference
Backoff or interpolation	Uses shorter-history evidence	The rule or weights still need evaluation
Shared neural parameters	Lets learning contribute across contexts	Still needs suitable data and training

Representative data should cover the situations we want to predict. Coverage alone is not sufficient: useful detailed situations also need enough observations. Duplicating the same examples does not create new independent evidence.

The relationship we care about

The issue is not that trigrams, or longer contexts, are inherently bad. It is the relationship between the detail the model distinguishes and the evidence available to estimate its probabilities.

More context can improve prediction when the distinction matters. With weak evidence, it can produce unreliable estimates, zero probabilities, or missing rows.

We select context lengths and smoothing choices using held-out validation data. There is no universal context length where performance must become worse.

7. Backoff: keep a backup

Return to the original training corpus: **anna and ava**. Our introductory rule is:

Use the trigram row if it exists. Otherwise, use the bigram row for the last token. If that row is also missing, use a uniform distribution over the four allowed outcomes.

Missing context: vn

The trigram never saw vn. Drop its oldest token, v, and try the shorter context n. That bigram row has two observations: n once and a once.

Next token	Backoff probability after vn
a	1/2
n	1/2
v	0
E	0

The reasoning is: "I do not know what follows vn, so I will use what I know about what follows n."

For generation, sample from this distribution. For evaluation, use the probability assigned to the actual next token.

Existing context, unseen target: an -> a

The an row exists, with one observation of n. Our rule uses it, giving n probability one and a probability zero. It does **not** fall back just because a particular target has zero count.

The decision is based on whether the context row exists, before knowing which next token will be evaluated or sampled.

What this rule does and does not do

It repairs missing rows by consulting broader evidence. It still permits zero probabilities inside seen rows. In our toy, every ordinary character has a bigram row, so the final uniform fallback is defensive rather than needed for vn.

This is a simple whole-row fallback, not Katz backoff or a complete smoothing algorithm. More sophisticated backoff methods allocate probability more carefully. Applying smoothing to the chosen distributions is another way to address their zeros.

8. Interpolation: use a second opinion

Backoff switches distributions when needed. Interpolation lets different context lengths contribute to the prediction together, even when the longer row exists.

For history an, combine the trigram's answer after an with the bigram's answer after n. Here we apply **add-one smoothing to both models first** and give each half the influence.

Next token	Trigram after an	Bigram after n	Equal mixture
a	1/5	1/3	4/15
n	2/5	1/3	11/30
v	1/5	1/6	11/60
E	1/5	1/6	11/60

The trigram an row has one training count plus four added counts, giving total five. The bigram n row has two training counts plus four added counts, giving total six: a and n each get 2/6; v and E each get 1/6.

$$P_{\text{mix}}(a \mid an) = \frac{1}{2} \times \frac{1}{5} + \frac{1}{2} \times \frac{1}{3} = \frac{4}{15}$$

This raises a's probability from 20% to about 26.7%. The bigram supplies evidence that a has followed n, even though it never followed an in training.

The general rule

$$P_{\text{mix}}(t \mid ab) = \lambda P_{\text{tri}}(t \mid ab) + (1 - \lambda) P_{\text{bi}}(t \mid b)$$

Lambda is the trigram's weight, between zero and one. Nonnegative weights summing to one preserve normalization when the component rows each sum to one.

Why mix?

The specific context may capture an important distinction, while the broader context pools more evidence. Mixing can reduce dependence on a tiny exact-match sample. It can also dilute a useful specific pattern, so the weights should be evaluated on held-out data.

One-half is our teaching choice, not a proven best weight. Improving this one continuation does not establish a better whole-model score.

Smoothing and interpolation are separate: smoothing adjusts each row; interpolation combines rows. If two unsmoothed models both give a target zero, mixing them still gives that target zero.

9. What carries forward?

Backoff uses shorter history as a backup. Interpolation uses it as a second opinion. Both allow a prediction to benefit from evidence beyond its exact longer-context row.

Later, neural networks will share learned parameters across contexts. This is a broader form of information sharing, rather than an explicit lookup of a shorter count row.

Series step	What changes
Episode 02	Extend count-table context and understand sparse evidence
Episode 03	Learn bigram probabilities using weights and softmax
Episode 07	Learn embeddings and an MLP with shared parameters
Later episodes	Build toward a small decoder-only Transformer

A learned bigram weight table alone does not solve longer-context sparsity. Shared representations are a later step, and they still need appropriate data and training.

The coding companion tests the tradeoff

The coding episode will use the larger names dataset from Episode 01. We will build models with different context lengths, then inspect:

- Training loss: how well they predict names used to build the counts.
- Held-out loss: how well they predict names outside that training set.
- Rare and unseen contexts: how much evidence supports their predictions.

If training loss improves while held-out loss worsens, better fitting of the training data did not transfer to those held-out names. Sparse evidence is one mechanism to investigate. The experiment must allow longer context to help; it should not assume its conclusion.

Check your understanding

1. Why does a trigram use two START pads but still add only one END target per name?
2. How does a zero target count differ from an entirely unseen context?
3. Why can a longer context have equal or fewer matches, but never more than its suffix in the same dataset?
4. Why does a sparse dictionary not solve the evidence problem?
5. Does our backoff rule fall back for an $\rightarrow a$? Why?
6. Where do the $1/5$ and $1/3$ in our interpolation example come from?

Answers are worked through in this guide. Additional exercises and a separate answer key are in episodes/02_trigrams in the repository.

The core lesson: more context can capture useful detail, but each separate count row needs evidence.

References and optional detail

The core n-gram methods are established. Our tiny names, examples, diagrams, teaching weights, and series sequence are pedagogical choices. These references do not imply an endorsement of this series.

N-grams, smoothing, backoff and interpolation

Daniel Jurafsky and James H. Martin, Speech and Language Processing, Chapter 3: N-gram Language Models. The online draft can change; use the chapter and section titles to locate the material.

<https://web.stanford.edu/~jurafsky/slp3/3.pdf>

Stanley F. Chen and Joshua Goodman (1996), An Empirical Study of Smoothing Techniques for Language Modeling. Supports comparison of smoothing choices through empirical evaluation.

<https://aclanthology.org/P96-1041/>

Shared neural representations

Yoshua Bengio, Rejean Ducharme, Pascal Vincent and Christian Jauvin (2003), A Neural Probabilistic Language Model. Introduces distributed representations and a shared neural probability model.

<https://jmlr.org/papers/v3/bengio03a.html>

Educational implementation and dataset provenance

Andrej Karpathy, makemore. An educational character-level modeling project. The larger coding dataset comes from this repository; its license and pinned provenance are preserved in our repository. The theory uses the explicit tiny corpora printed in this guide.

<https://github.com/karpathy/makemore>

Why not allocate $(A + 1)$ to the power m contexts?

Treating START as another independent choice at every history position also creates invalid placements, such as aS. Our row formula counts only START prefixes followed by ordinary characters.

Why K probabilities have $K - 1$ free values

A row's probabilities must sum to one. Once $K - 1$ probabilities are chosen, the last is determined by the remaining probability mass. For example, if a two-outcome row gives one outcome 0.3, the other must be 0.7.

Cells therefore are not independent free parameters. This does not change the number of cells a naive full table stores.

Verification

Twelve automated checks cover the lab and lesson arithmetic. This is internal verification, not independent certification. Real-data test evaluation belongs to the coding release.